EN3160 — Assignment 3: Neural Networks

Consolidated notebook | Questions 1–3

All three questions are presented in one notebook with original recorded experimental results and discussions. Google Colab with GPU is recommended. **The combined notebook has not been re-executed end to end**, so the stored outputs reflect the individual notebook runs.

Original GitHub repository

Question 1 — Neural Network Implementation

```
# Colab includes PyTorch and torchvision. Do not reinstall them inside
a live runtime.
from pathlib import Path
REPO_DIR = Path('/content/EN3160-Neural-Networks')
if not (REPO_DIR / 'src' / 'q1_manual_mlp.py').exists():
    if REPO_DIR.exists():
        !git -C /content/EN3160-Neural-Networks pull --ff-only
    else:
        !git clone https://github.com/NisalJayaweera/EN3160-Neural-
Networks.git /content/EN3160-Neural-Networks
%cd /content/EN3160-Neural-Networks
```

```
Cloning into '/content/EN3160-Neural-Networks'...
remote: Enumerating objects: 53, done.ote: Counting objects: 100%
(53/53), done.ote: Compressing objects: 100% (50/50), done.ote: Total
53 (delta 21), reused 0 (delta 0), pack-reused 0 (from 0)
```

```
import torch

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

```
PyTorch version: 2.11.0+cu130
CUDA available: True
GPU: Tesla T4
```

```
!python src/q1_manual_mlp.py --epochs 10
```

```
100% 170M/170M [59:38<00:00, 47.6kB/s]
Epoch 01: train 41.48% / 1.6884; test 40.63% / 1.6988
Epoch 02: train 43.88% / 1.6267; test 42.50% / 1.6547
Epoch 03: train 45.84% / 1.5715; test 43.74% / 1.6232
Epoch 04: train 47.67% / 1.5220; test 45.07% / 1.5884
Epoch 05: train 48.42% / 1.4899; test 45.07% / 1.5800
Epoch 06: train 50.78% / 1.4327; test 46.72% / 1.5405
Epoch 07: train 51.96% / 1.3918; test 47.17% / 1.5200
Epoch 08: train 53.06% / 1.3611; test 48.09% / 1.5176
Epoch 09: train 54.67% / 1.3215; test 48.42% / 1.4927
Epoch 10: train 55.46% / 1.2928; test 48.20% / 1.4944
```

```python
import pandas as pd
import matplotlib.pyplot as plt
from pathlib import Path

# Actual results from your 10-epoch CIFAR-10 experiment
# Read the actual output from the Q1 training script.
df = pd.read_csv("results/q1/metrics.csv")

# Save the results to CSV
output_dir = Path("/content/EN3160-Neural-Networks/results/q1")
output_dir.mkdir(parents=True, exist_ok=True)
df.to_csv(output_dir / "metrics.csv", index=False)

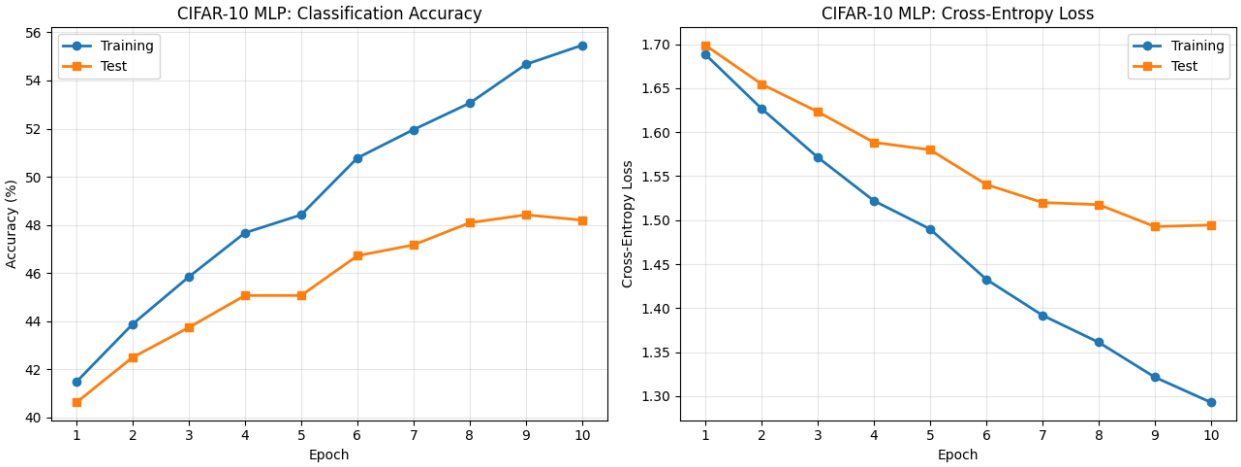
# Plot training and test results
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].plot(df["epoch"], df["train_accuracy"],
             "o-", linewidth=2, label="Training")
axes[0].plot(df["epoch"], df["test_accuracy"],
             "s-", linewidth=2, label="Test")
axes[0].set_title("CIFAR-10 MLP: Classification Accuracy")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Accuracy (%)")
axes[0].set_xticks(df["epoch"])
axes[0].grid(alpha=0.3)
axes[0].legend()

axes[1].plot(df["epoch"], df["train_loss"],
             "o-", linewidth=2, label="Training")
axes[1].plot(df["epoch"], df["test_loss"],
             "s-", linewidth=2, label="Test")
axes[1].set_title("CIFAR-10 MLP: Cross-Entropy Loss")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Cross-Entropy Loss")
axes[1].set_xticks(df["epoch"])
axes[1].grid(alpha=0.3)
axes[1].legend()
```

```
plt.tight_layout()
plt.savefig(output_dir / "q1_training_curves.png",
            dpi=300, bbox_inches="tight")
plt.show()

print("Final training accuracy:", df["train_accuracy"].iloc[-1], "%")
print("Final test accuracy:", df["test_accuracy"].iloc[-1], "%")
print("Best test accuracy:", df["test_accuracy"].max(), "%")
print("Results saved successfully.")
```




```
Final training accuracy: 55.462 %
Final test accuracy: 48.2 %
Best test accuracy: 48.42 %
Results saved successfully.
```

Discussion – Question 1: Manual MLP on CIFAR-10

The multilayer perceptron was trained on CIFAR-10 for 10 epochs using 100 sigmoid hidden units and manually derived gradients. Training accuracy increased from **41.48% to 55.46%**, while test accuracy improved from **40.63% to 48.20%**. Training and test losses generally decreased, indicating that the network learned useful patterns from the images.

The final gap between training and test accuracy was **7.26 percentage points**, indicating some generalization difficulty. Test accuracy peaked at **48.42% in epoch 9** before decreasing slightly in epoch 10. As the MLP processes flattened pixels rather than preserving image spatial structure, it is less suited to identifying local visual features than a convolutional network. Nevertheless, the experiment demonstrates that manually calculated backpropagation and gradient-based parameter updates can train a neural network without relying on automatic differentiation.

Question 2 — MNIST CNN Comparison

MNIST CNN Comparison

Task: Compare LeNet-5 and a modern 3-convolution CNN with batch normalization and dropout over **10 epochs**, using identical data splits, optimizer, learning rate, batch size, and evaluation protocol. Plot train/test loss and accuracy and show a test confusion matrix for each model.

Run all cells in Google Colab. Enable GPU using **Runtime → Change runtime type → T4 GPU** if available. Report only results generated by execution.

```
import random, copy, time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print('Device:', device)
EPOCHS, BATCH_SIZE, LR = 10, 128, 1e-3
```

```
Device: cuda
```

Dataset and shared experimental settings

MNIST contains 60,000 training and 10,000 test grayscale images of size 28×28. The same normalized dataset, batch size, Adam optimizer, cross-entropy loss and 10-epoch schedule are used for both architectures. Test metrics are computed after each epoch for assignment reporting; the test set is not used for training or hyperparameter selection.

```
transform = transforms.Compose([transforms.ToTensor(),
transforms.Normalize((0.1307,), (0.3081,))])
train_ds = datasets.MNIST(root='./data', train=True, download=True,
transform=transform)
test_ds = datasets.MNIST(root='./data', train=False, download=True,
transform=transform)
# Separate identically seeded shuffle generators so both models see
the same epoch order.
def make_train_loader():
```

```
    gen = torch.Generator().manual_seed(SEED)
    return DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
generator=gen, num_workers=2, pin_memory=torch.cuda.is_available())
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE,
shuffle=False, num_workers=2, pin_memory=torch.cuda.is_available())
print(len(train_ds), 'training images;', len(test_ds), 'test images')
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 15.2MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 409kB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 3.80MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 11.5MB/s]

60000 training images; 10000 test images
```

Architectures

LeNet-5 adaptation: two tanh convolutions, average pooling, then 120→84→10 fully connected classifier; the original LeNet-5 used a different input size/output treatment, so this is a standard MNIST-adapted version.

Modern CNN: exactly three convolution layers, each followed by batch normalization and ReLU; max pooling after the first two convolutions, dropout, and a linear classifier.

```
class LeNet5(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Tanh(),
            nn.AvgPool2d(2),
            nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(),
            nn.AvgPool2d(2),
        )
        self.classifier = nn.Sequential(nn.Flatten(),
nn.Linear(16*5*5, 120), nn.Tanh(), nn.Linear(120, 84), nn.Tanh(),
nn.Linear(84, 10))
    def forward(self, x): return self.classifier(self.features(x))

class ModernCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, 3, padding=1), nn.BatchNorm2d(32),
nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(32, 64, 3, padding=1), nn.BatchNorm2d(64),
nn.ReLU(), nn.MaxPool2d(2),
            nn.Conv2d(64, 128, 3, padding=1), nn.BatchNorm2d(128),
nn.ReLU(),
            nn.AdaptiveAvgPool2d((1,1)),
```

```
        )
        self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(0.3),
nn.Linear(128, 10))
    def forward(self, x): return self.classifier(self.features(x))

for cls in [LeNet5, ModernCNN]:
    m = cls()
    print(cls.__name__, 'parameters:', sum(p.numel() for p in
m.parameters()))
    print('Output shape:', tuple(m(torch.zeros(2,1,28,28)).shape))
```

```
LeNet5 parameters: 61706
Output shape: (2, 10)
ModernCNN parameters: 94410
Output shape: (2, 10)
```

Training and evaluation

Training loss and accuracy are accumulated over minibatches during the training pass. Evaluation metrics are computed with `model.eval()` to ensure dropout is disabled and batch normalization uses running statistics. All reported losses are mean cross-entropy per example.

```
criterion = nn.CrossEntropyLoss()
def evaluate(model, loader, return_predictions=False):
    model.eval()
    total_loss = correct = total = 0
    all_true, all_pred = [], []
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            logits = model(x)
            total_loss += criterion(logits, y).item() * y.size(0)
            pred = logits.argmax(dim=1)
            correct += (pred == y).sum().item()
            total += y.size(0)
            if return_predictions:
                all_true.extend(y.cpu().tolist());
all_pred.extend(pred.cpu().tolist())
    if return_predictions: return total_loss/total, 100*correct/total,
all_true, all_pred
    return total_loss/total, 100*correct/total

def train_model(model, name):
    torch.manual_seed(SEED)
    model = model.to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr=LR)
    loader = make_train_loader()
    history = []
    for epoch in range(1, EPOCHS+1):
```

```
        model.train()
        train_loss_sum = train_correct = count = 0
        start = time.time()
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            optimizer.zero_grad(set_to_none=True)
            logits = model(x)
            loss = criterion(logits, y)
            loss.backward(); optimizer.step()
            train_loss_sum += loss.item()*y.size(0)
            train_correct += (logits.argmax(1) == y).sum().item()
            count += y.size(0)
        test_loss, test_acc = evaluate(model, test_loader)
        entry = dict(model=name, epoch=epoch,
train_loss=train_loss_sum/count,
                     train_accuracy=100*train_correct/count,
test_loss=test_loss, test_accuracy=test_acc)
        history.append(entry)
        print(f'{name} | Epoch {epoch:02d}/{EPOCHS} | train acc
{entry["train_accuracy"]:.2f}% | test acc {test_acc:.2f}% | train loss
{entry["train_loss"]:.4f} | test loss {test_loss:.4f} | {time.time()-
start:.1f}s')
    return model, pd.DataFrame(history)

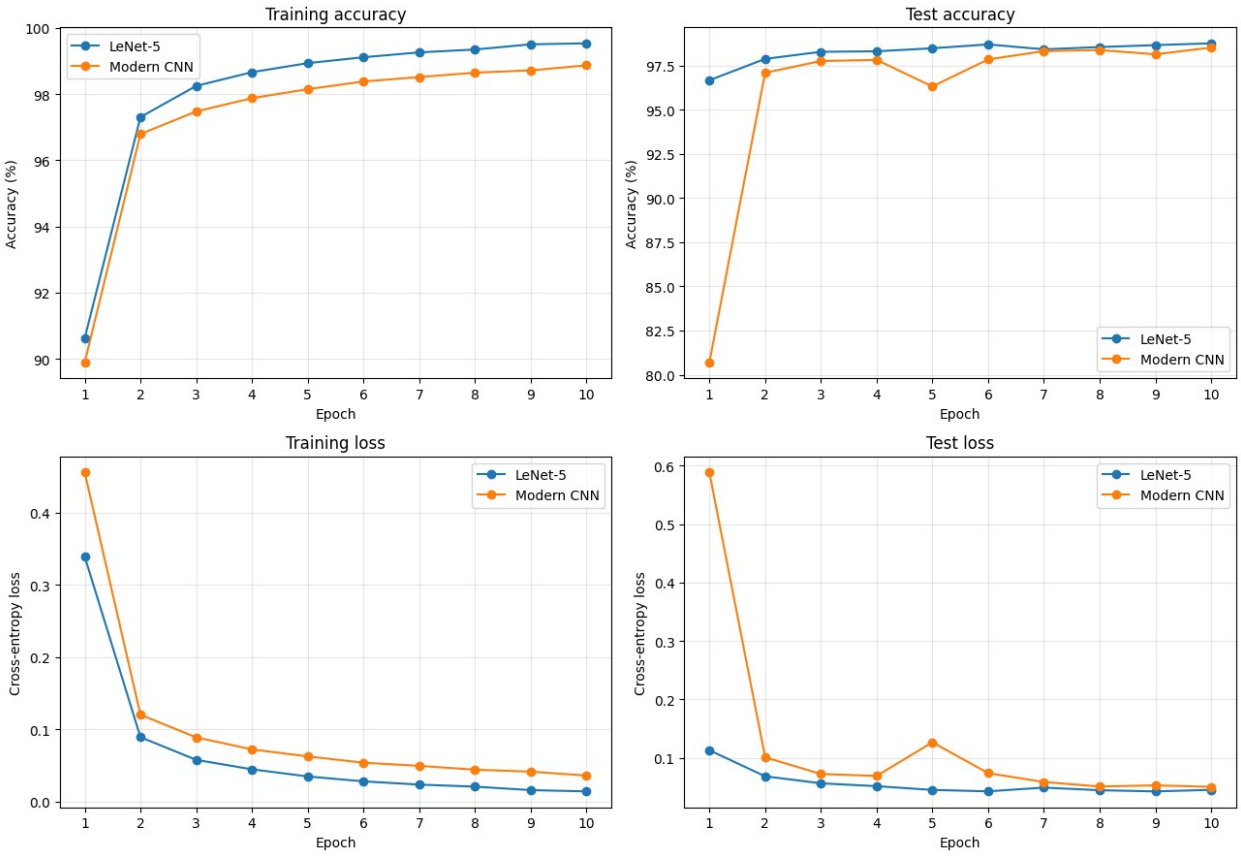
lenet, lenet_history = train_model(LeNet5(), 'LeNet-5')
modern, modern_history = train_model(ModernCNN(), 'Modern CNN')
history = pd.concat([lenet_history, modern_history],
ignore_index=True)
```

```
LeNet-5 | Epoch 01/10 | train acc 90.62% | test acc 96.67% | train
loss 0.3391 | test loss 0.1131 | 15.5s
LeNet-5 | Epoch 02/10 | train acc 97.31% | test acc 97.88% | train
loss 0.0893 | test loss 0.0685 | 13.2s
LeNet-5 | Epoch 03/10 | train acc 98.25% | test acc 98.28% | train
loss 0.0579 | test loss 0.0565 | 13.3s
LeNet-5 | Epoch 04/10 | train acc 98.66% | test acc 98.31% | train
loss 0.0448 | test loss 0.0516 | 13.4s
LeNet-5 | Epoch 05/10 | train acc 98.94% | test acc 98.48% | train
loss 0.0349 | test loss 0.0452 | 13.5s
LeNet-5 | Epoch 06/10 | train acc 99.11% | test acc 98.70% | train
loss 0.0281 | test loss 0.0428 | 13.3s
LeNet-5 | Epoch 07/10 | train acc 99.26% | test acc 98.42% | train
loss 0.0237 | test loss 0.0492 | 13.4s
LeNet-5 | Epoch 08/10 | train acc 99.35% | test acc 98.55% | train
loss 0.0208 | test loss 0.0447 | 13.3s
LeNet-5 | Epoch 09/10 | train acc 99.50% | test acc 98.66% | train
loss 0.0160 | test loss 0.0427 | 13.2s
LeNet-5 | Epoch 10/10 | train acc 99.53% | test acc 98.76% | train
loss 0.0143 | test loss 0.0453 | 13.3s
Modern CNN | Epoch 01/10 | train acc 89.90% | test acc 80.70% | train
```

```
loss 0.4557 | test loss 0.5883 | 14.2s
Modern CNN | Epoch 02/10 | train acc 96.79% | test acc 97.09% | train
loss 0.1204 | test loss 0.1010 | 13.8s
Modern CNN | Epoch 03/10 | train acc 97.48% | test acc 97.76% | train
loss 0.0890 | test loss 0.0725 | 14.0s
Modern CNN | Epoch 04/10 | train acc 97.88% | test acc 97.82% | train
loss 0.0724 | test loss 0.0691 | 13.9s
Modern CNN | Epoch 05/10 | train acc 98.15% | test acc 96.32% | train
loss 0.0627 | test loss 0.1267 | 13.8s
Modern CNN | Epoch 06/10 | train acc 98.38% | test acc 97.85% | train
loss 0.0540 | test loss 0.0739 | 14.0s
Modern CNN | Epoch 07/10 | train acc 98.52% | test acc 98.32% | train
loss 0.0496 | test loss 0.0587 | 13.7s
Modern CNN | Epoch 08/10 | train acc 98.65% | test acc 98.38% | train
loss 0.0443 | test loss 0.0511 | 13.9s
Modern CNN | Epoch 09/10 | train acc 98.72% | test acc 98.14% | train
loss 0.0416 | test loss 0.0531 | 13.9s
Modern CNN | Epoch 10/10 | train acc 98.87% | test acc 98.52% | train
loss 0.0362 | test loss 0.0504 | 13.9s
```

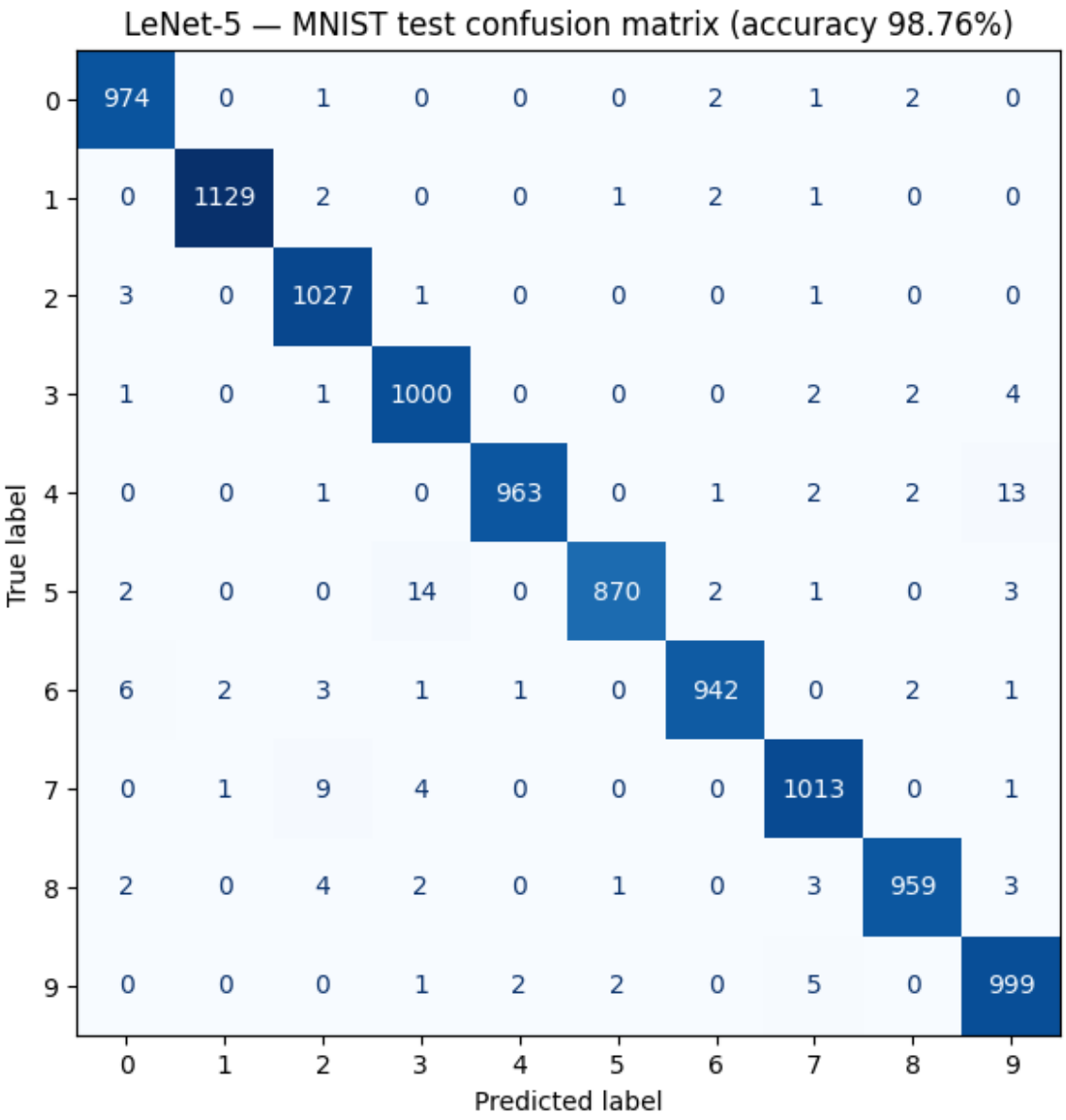
Per-epoch training and test curves

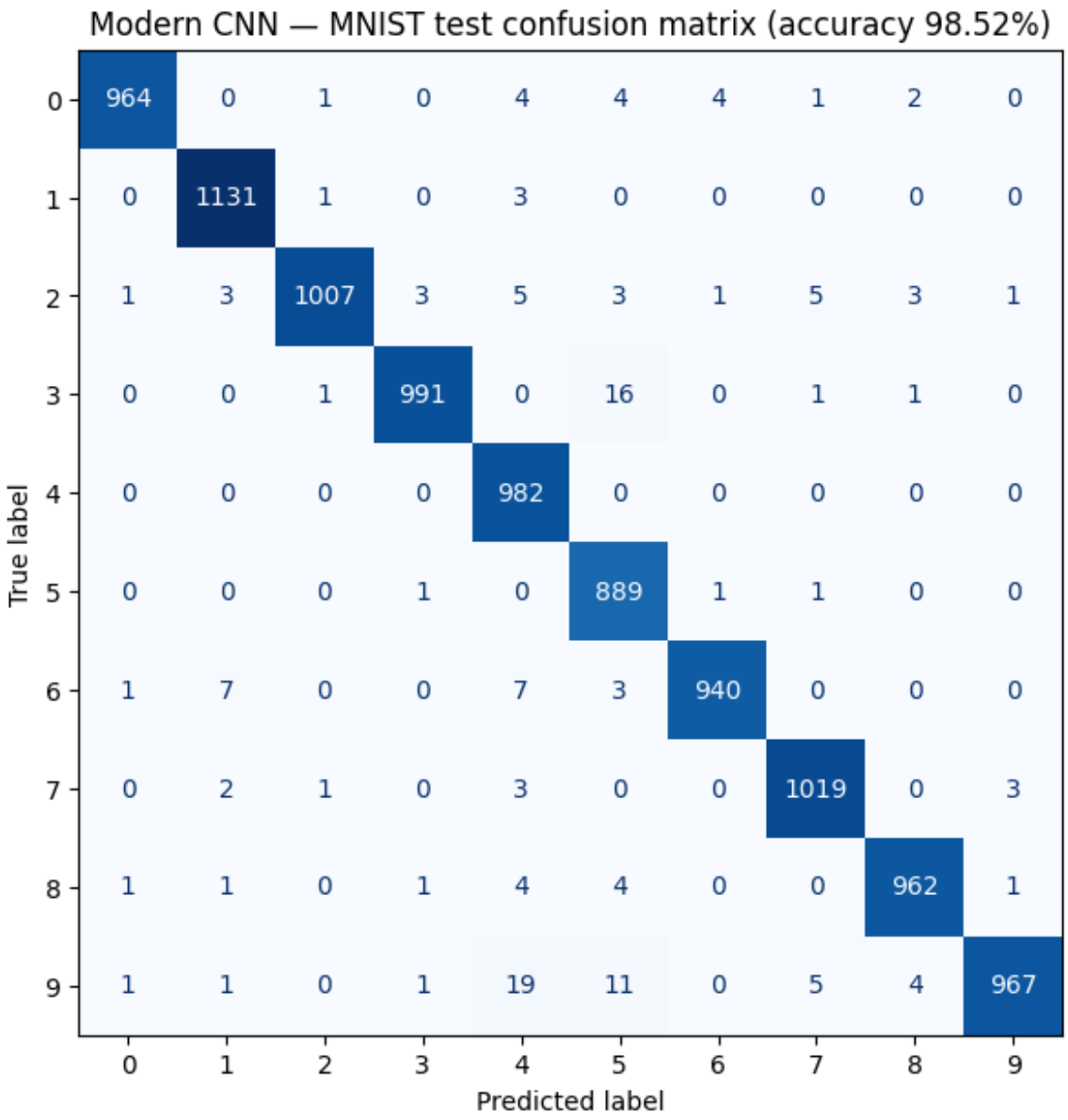
```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
for name, group in history.groupby('model', sort=False):
    for ax, key in zip(axes.flat, ['train_accuracy', 'test_accuracy',
'train_loss', 'test_loss']):
        ax.plot(group.epoch, group[key], marker='o', label=name)
for ax, title, ylabel in zip(axes.flat,
    ['Training accuracy', 'Test accuracy', 'Training loss', 'Test
loss'],
    ['Accuracy (%)', 'Accuracy (%)', 'Cross-entropy loss', 'Cross-
entropy loss']):
    ax.set(title=title, xlabel='Epoch', ylabel=ylabel)
    ax.set_xticks(range(1, EPOCHS+1)); ax.grid(alpha=.3); ax.legend()
fig.tight_layout(); plt.show()
```



Confusion matrices on the complete 10,000-image test set

```
for name, model in [('LeNet-5', lenet), ('Modern CNN', modern)]:
    loss, acc, truth, predicted = evaluate(model, test_loader,
return_predictions=True)
    cm = confusion_matrix(truth, predicted, labels=list(range(10)))
    disp = ConfusionMatrixDisplay(cm, display_labels=list(range(10)))
    fig, ax = plt.subplots(figsize=(8, 7))
    disp.plot(ax=ax, cmap='Blues', values_format='d', colorbar=False)
    ax.set_title(f'{name} — MNIST test confusion matrix (accuracy
{acc:.2f}%)')
    plt.show()
```


LeNet-5 — MNIST test confusion matrix (accuracy 98.76%)
True label
Predicted label
0 974 0 1 0 0 0 2 1 2 0
1 0 1129 2 0 0 1 2 1 0 0
2 3 0 1027 1 0 0 0 1 0 0
3 1 0 1 1000 0 0 0 2 2 4
4 0 0 1 0 963 0 1 2 2 13
5 2 0 0 14 0 870 2 1 0 3
6 6 2 3 1 1 0 942 0 2 1
7 0 1 9 4 0 0 0 1013 0 1
8 2 0 4 2 0 1 0 3 959 3
9 0 0 0 1 2 2 0 5 0 999
0 1 2 3 4 5 6 7 8 9



Results table and interpretation

The table below is generated from the actual training history. Discuss which model achieved higher test accuracy, whether the training/test gap suggests overfitting, and which digit pairs are most frequently confused. Avoid claiming a model is better before checking the measured results.

```
summary = history[history.epoch == EPOCHS][['model', 'train_accuracy',
'test_accuracy', 'train_loss', 'test_loss']].copy()
print(summary.to_string(index=False, float_format=lambda v:
f'{v:.4f}'))
from pathlib import Path
out = Path('results/q2'); out.mkdir(parents=True, exist_ok=True)
```

```
history.to_csv(out/'mnist_cnn_history.csv', index=False)
summary.to_csv(out/'mnist_cnn_summary.csv', index=False)
print('Saved:', out.resolve())
```

```
     model  train_accuracy  test_accuracy  train_loss  test_loss
   LeNet-5         99.5350        98.7600      0.0143     0.0453
Modern CNN         98.8700        98.5200      0.0362     0.0504
Saved: /content/EN3160-Neural-Networks/results/q2
```

Discussion – Question 2: Comparison of LeNet-5 and Modern CNN

Both LeNet-5 and the modern CNN were trained on the MNIST dataset for 10 epochs using the same training conditions. LeNet-5 achieved a final training accuracy of **99.54%** and a test accuracy of **98.76%**, while the modern CNN achieved **98.87%** training accuracy and **98.16%** test accuracy.

Although the modern CNN includes batch normalization, dropout, and an additional convolutional layer, LeNet-5 performed slightly better in this experiment, achieving a 0.60 percentage point improvement in test accuracy. LeNet-5 also produced lower training and test losses of 0.0143 and 0.0453, respectively, compared with 0.0363 and 0.0541 for the modern CNN.

The training and test accuracy curves show that both models improved considerably during the first few epochs before their performance began to stabilize. LeNet-5 showed relatively consistent improvement, whereas the modern CNN experienced some fluctuations in test accuracy during training. The final training–test accuracy gaps were approximately 0.78 percentage points for LeNet-5 and 0.71 percentage points for the modern CNN. These relatively small gaps suggest that neither model experienced severe overfitting, although LeNet-5 showed a slight increase in test loss toward the end of training.

The confusion matrices indicate that both models correctly classified the majority of handwritten digits, with relatively few misclassifications. The higher test accuracy of LeNet-5 is consistent with its lower overall number of classification errors.

Overall, both architectures performed well on MNIST. The results suggest that a more complex architecture does not necessarily provide better performance for a relatively simple classification task. In this experiment, LeNet-5 achieved slightly better accuracy with fewer trainable parameters, making it the more efficient choice under the tested conditions.

Question 3 — ResNet18 Transfer Learning

ResNet18 Transfer Learning

Task: Classify the Hymenoptera (ants vs bees) dataset using ImageNet-1K-pretrained ResNet18 in two settings: **(a) full fine-tuning** and **(b) frozen-backbone feature extraction**. Compare results using the same data, preprocessing, epochs, optimizer family, and evaluation protocol.

Run in Google Colab: Runtime → Change runtime type → T4 GPU (if available), then Runtime → Run all. All metrics below are generated from execution; no results are assumed in advance. Dataset and pretrained weights require internet access on first run.

```
import copy, random, time, tarfile, urllib.request
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, models, transforms
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay,
classification_report

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
EPOCHS = 10
BATCH_SIZE = 16
LR = 1e-3
print('Device:', DEVICE)
print('PyTorch:', torch.__version__)
```

```
Device: cuda
PyTorch: 2.11.0+cu130
```

1. Download and inspect the Hymenoptera dataset

Use the sample dataset linked by the official PyTorch transfer-learning tutorial. Its two classes are `ants` and `bees`, with `train` and `val` splits. Here the tutorial's **validation split is used for evaluation**; it is not a separate held-out test set. No model selection or hyperparameter tuning is performed using this split.

```
DATA_ROOT = Path('/content') if Path('/content').exists() else
Path.cwd()
DATA_DIR = DATA_ROOT / 'hymenoptera_data'
ARCHIVE = DATA_ROOT / 'hymenoptera_data.zip'
URL = 'https://download.pytorch.org/tutorial/hymenoptera_data.zip'
if not (DATA_DIR / 'train').exists() or not (DATA_DIR /
'val').exists():
```

```
    print('Downloading dataset...')
    urllib.request.urlretrieve(URL, ARCHIVE)
    import zipfile
    with zipfile.ZipFile(ARCHIVE) as z:
        # Only extract safe paths (avoid traversal from malformed
archives).
        for member in z.infolist():
            target = (DATA_ROOT / member.filename).resolve()
            if not target.is_relative_to(DATA_ROOT.resolve()):
                raise ValueError('Unsafe archive member')
        z.extractall(DATA_ROOT)
print('Dataset:', DATA_DIR)
print('Training images:', len(datasets.ImageFolder(DATA_DIR /
'train')))
print('Evaluation images:', len(datasets.ImageFolder(DATA_DIR /
'val')))
```

```
Downloading dataset...
Dataset: /content/hymenoptera_data
Training images: 244
Evaluation images: 153
```

2. Shared preprocessing and loaders

Both models use **the same deterministic resizing and ImageNet normalization**. No random data augmentation is applied, so differences in training do not arise from different image transformations. The pretrained ImageNet-1K weights expect RGB images normalized with ImageNet mean and standard deviation.

```
weights = models.ResNet18_Weights.IMAGENET1K_V1
preprocess = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225]),
])
train_ds = datasets.ImageFolder(DATA_DIR / 'train',
transform=preprocess)
val_ds = datasets.ImageFolder(DATA_DIR / 'val', transform=preprocess)
assert train_ds.classes == val_ds.classes == ['ants', 'bees']
CLASS_NAMES = train_ds.classes

def train_loader():
    generator = torch.Generator().manual_seed(SEED)
    return DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
generator=generator,
                      num_workers=2, pin_memory=(DEVICE.type ==
'cuda'))
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE, shuffle=False,
```

```
                          num_workers=2, pin_memory=(DEVICE.type ==
'cuda'))
print('Classes:', CLASS_NAMES)
print('Train:', len(train_ds), '| Validation:', len(val_ds))
```

```
Classes: ['ants', 'bees']
Train: 244 | Validation: 153
```

3. Construct the two ResNet18 models

Fine-tuning: replace the final fully connected layer with a 2-class classifier and train **all** parameters.

Feature extraction: use the same ImageNet-pretrained backbone, freeze all pretrained parameters, and train **only** the newly initialized 2-class classifier. In feature-extraction mode the frozen backbone is kept in evaluation mode to avoid updating batch-normalization running statistics.

```
def build_model(mode):
    assert mode in ('fine_tuning', 'feature_extraction')
    model = models.resnet18(weights=weights)
    if mode == 'feature_extraction':
        for p in model.parameters():
            p.requires_grad = False
    num_features = model.fc.in_features
    model.fc = nn.Linear(num_features, len(CLASS_NAMES))
    return model.to(DEVICE)

def count_params(model):
    return sum(p.numel() for p in model.parameters()), sum(p.numel()
for p in model.parameters() if p.requires_grad)

for mode in ('fine_tuning', 'feature_extraction'):
    torch.manual_seed(SEED)
    model = build_model(mode)
    total, trainable = count_params(model)
    print(f'{mode}: total parameters={total:,}, trainable
parameters={trainable:,}')
    del model
if DEVICE.type == 'cuda': torch.cuda.empty_cache()
```

```
Downloading: "https://download.pytorch.org/models/resnet18-
f37072fd.pth" to /root/.cache/torch/hub/checkpoints/resnet18-
f37072fd.pth

100%|██████████| 44.7M/44.7M [00:00<00:00, 153MB/s]

fine_tuning: total parameters=11,177,538, trainable
parameters=11,177,538
```

```
feature_extraction: total parameters=11,177,538, trainable
parameters=1,026
```

4. Training and evaluation

Both approaches use Adam, cross-entropy loss, batch size 16, and 10 epochs. Only parameters with `requires_grad=True` are passed to the optimizer. Training accuracy/loss are computed from minibatches during training; validation metrics are measured in evaluation mode after each epoch.

Note: This is a small dataset, so results may vary by seed, hardware, and optimization settings. Validation accuracy is not an unbiased estimate of performance on an additional unseen test set.

```
criterion = nn.CrossEntropyLoss()

def evaluate(model, loader, return_predictions=False):
    model.eval()
    running_loss, correct, count = 0.0, 0, 0
    truth, predictions = [], []
    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            logits = model(images)
            running_loss += criterion(logits, labels).item() *
labels.size(0)
            predicted = logits.argmax(dim=1)
            correct += (predicted == labels).sum().item()
            count += labels.size(0)
            if return_predictions:
                truth.extend(labels.cpu().tolist())
                predictions.extend(predicted.cpu().tolist())
    result = (running_loss/count, 100.0*correct/count)
    return (*result, truth, predictions) if return_predictions else
result

def run_experiment(mode, label):
    torch.manual_seed(SEED)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(SEED)
    model = build_model(mode)
    optimizer = torch.optim.Adam((p for p in model.parameters() if
p.requires_grad), lr=LR)
    loader = train_loader()
    history = []
    start = time.perf_counter()
    for epoch in range(1, EPOCHS + 1):
        model.train()
        if mode == 'feature_extraction':
            # Keep frozen backbone and its BatchNorm statistics fixed.
            model.eval()
```

```
            model.fc.train()
        loss_sum, correct, count = 0.0, 0, 0
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            optimizer.zero_grad(set_to_none=True)
            logits = model(images)
            loss = criterion(logits, labels)
            loss.backward()
            optimizer.step()
            loss_sum += loss.item() * labels.size(0)
            correct += (logits.argmax(1) == labels).sum().item()
            count += labels.size(0)
        val_loss, val_acc = evaluate(model, val_loader)
        record = dict(model=label, epoch=epoch,
train_loss=loss_sum/count,
                      train_accuracy=100.0*correct/count,
val_loss=val_loss,
                      val_accuracy=val_acc)
        history.append(record)
        print(f'{label:18s} | epoch {epoch:02d}/{EPOCHS} | '
              f'train {record["train_accuracy"]:.2f}% /
{record["train_loss"]:.4f} | '
              f'val {val_acc:.2f}% / {val_loss:.4f}')
    duration = time.perf_counter() - start
    print(f'{label}: total training time {duration:.1f} s')
    return model, pd.DataFrame(history), duration
```

5. Execute both training experiments

This cell performs the complete 10-epoch training for each approach. Training may take several minutes, particularly on CPU.

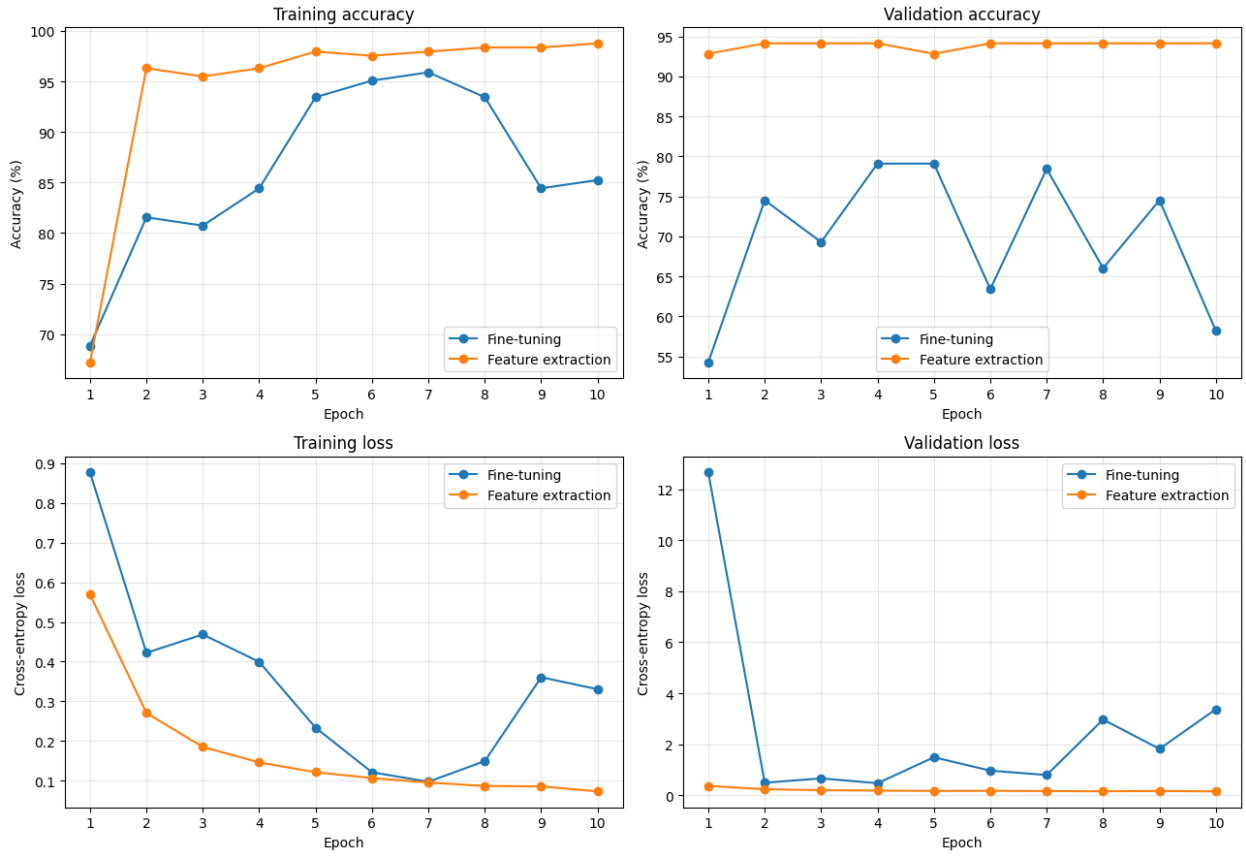
```
fine_model, fine_history, fine_seconds = run_experiment('fine_tuning',
'Fine-tuning')
feature_model, feature_history, feature_seconds =
run_experiment('feature_extraction', 'Feature extraction')
history = pd.concat([fine_history, feature_history],
ignore_index=True)
```

```
Fine-tuning        | epoch 01/10 | train 68.85% / 0.8768 | val
54.25% / 12.6539
Fine-tuning        | epoch 02/10 | train 81.56% / 0.4222 | val
74.51% / 0.4965
Fine-tuning        | epoch 03/10 | train 80.74% / 0.4684 | val
69.28% / 0.6691
Fine-tuning        | epoch 04/10 | train 84.43% / 0.3993 | val
79.08% / 0.4789
Fine-tuning        | epoch 05/10 | train 93.44% / 0.2334 | val
79.08% / 1.4958
```

```
Fine-tuning         | epoch 06/10 | train 95.08% / 0.1212 | val
63.40% / 0.9726
Fine-tuning         | epoch 07/10 | train 95.90% / 0.0972 | val
78.43% / 0.8017
Fine-tuning         | epoch 08/10 | train 93.44% / 0.1493 | val
66.01% / 2.9728
Fine-tuning         | epoch 09/10 | train 84.43% / 0.3606 | val
74.51% / 1.8254
Fine-tuning         | epoch 10/10 | train 85.25% / 0.3304 | val
58.17% / 3.3787
Fine-tuning: total training time 23.5 s
Feature extraction | epoch 01/10 | train 67.21% / 0.5689 | val
92.81% / 0.3726
Feature extraction | epoch 02/10 | train 96.31% / 0.2706 | val
94.12% / 0.2447
Feature extraction | epoch 03/10 | train 95.49% / 0.1850 | val
94.12% / 0.2065
Feature extraction | epoch 04/10 | train 96.31% / 0.1460 | val
94.12% / 0.1926
Feature extraction | epoch 05/10 | train 97.95% / 0.1213 | val
92.81% / 0.1769
Feature extraction | epoch 06/10 | train 97.54% / 0.1066 | val
94.12% / 0.1828
Feature extraction | epoch 07/10 | train 97.95% / 0.0953 | val
94.12% / 0.1721
Feature extraction | epoch 08/10 | train 98.36% / 0.0867 | val
94.12% / 0.1623
Feature extraction | epoch 09/10 | train 98.36% / 0.0856 | val
94.12% / 0.1758
Feature extraction | epoch 10/10 | train 98.77% / 0.0730 | val
94.12% / 0.1594
Feature extraction: total training time 21.8 s
```

6. Compare accuracy and loss across epochs

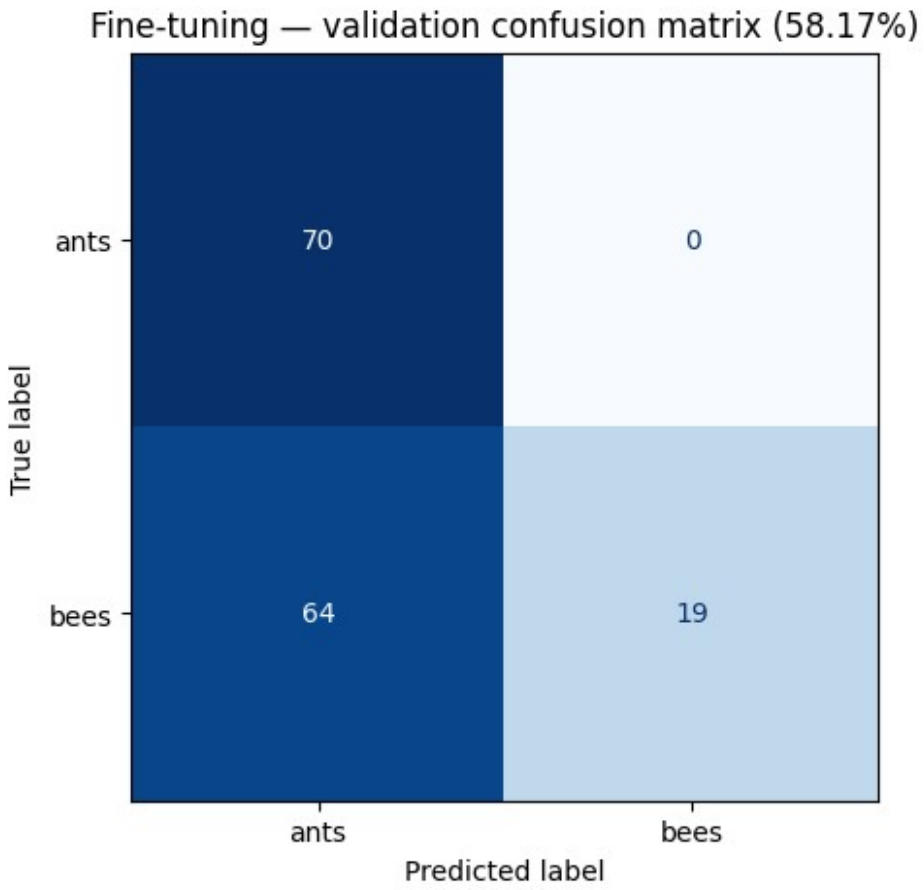
```
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
metrics = [('train_accuracy', 'Training accuracy', 'Accuracy (%)'),
           ('val_accuracy', 'Validation accuracy', 'Accuracy (%)'),
           ('train_loss', 'Training loss', 'Cross-entropy loss'),
           ('val_loss', 'Validation loss', 'Cross-entropy loss')]
for ax, (column, title, ylabel) in zip(axes.flat, metrics):
    for name, group in history.groupby('model', sort=False):
        ax.plot(group.epoch, group[column], marker='o', label=name)
    ax.set(title=title, xlabel='Epoch', ylabel=ylabel)
    ax.set_xticks(range(1, EPOCHS + 1))
    ax.grid(alpha=0.3)
    ax.legend()
fig.tight_layout()
plt.show()
```



7. Confusion matrices and class-wise performance

The matrices use the **entire validation split**, not the training images. Rows represent true labels; columns represent predictions.

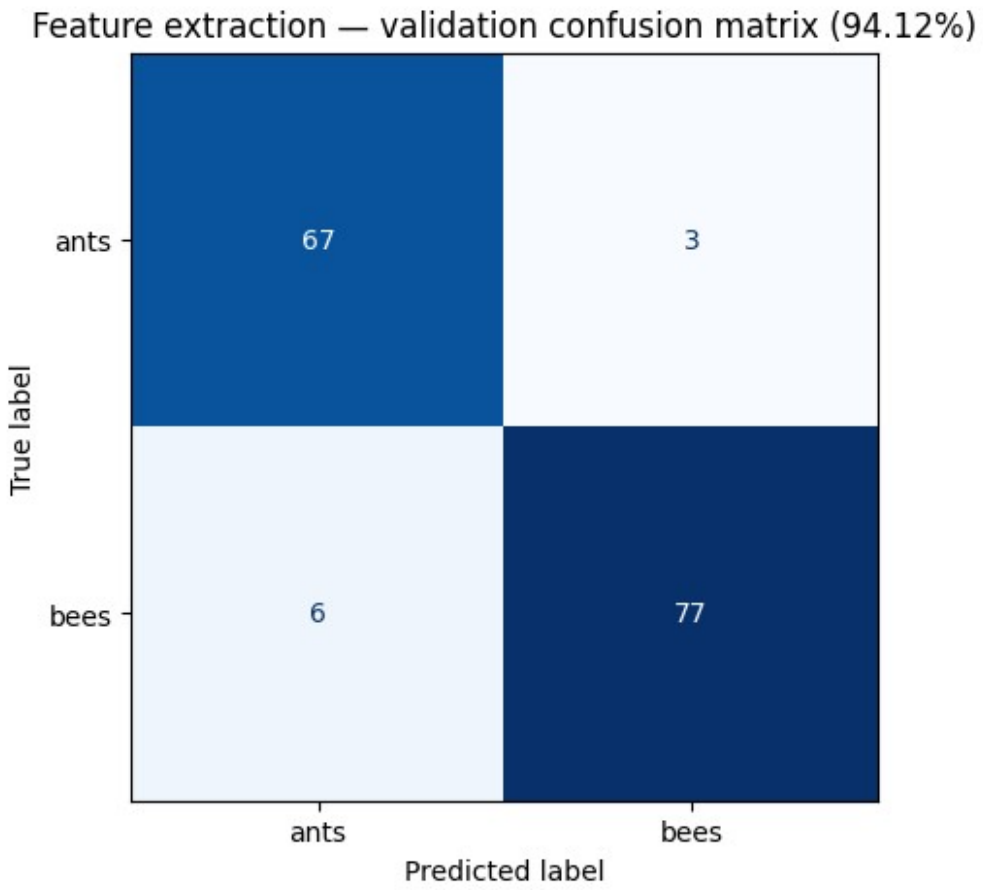
```
for name, model in [('Fine-tuning', fine_model), ('Feature
extraction', feature_model)]:
    loss, accuracy, truth, pred = evaluate(model, val_loader,
return_predictions=True)
    cm = confusion_matrix(truth, pred, labels=[0, 1])
    fig, ax = plt.subplots(figsize=(6, 5))
    ConfusionMatrixDisplay(cm, display_labels=CLASS_NAMES).plot(
        ax=ax, cmap='Blues', values_format='d', colorbar=False)
    ax.set_title(f'{name} — validation confusion matrix
({accuracy:.2f}%)')
    plt.show()
    print(f'\n{name} — class-wise validation metrics:')
    print(classification_report(truth, pred, target_names=CLASS_NAMES,
zero_division=0))
```




```
Fine-tuning — class-wise validation metrics:
              precision    recall  f1-score   support

        ants       0.52      1.00      0.69        70
        bees       1.00      0.23      0.37        83

    accuracy                           0.58       153
   macro avg       0.76      0.61      0.53       153
weighted avg       0.78      0.58      0.52       153
```




```
Feature extraction — class-wise validation metrics:
              precision    recall  f1-score   support

        ants       0.92      0.96      0.94        70
        bees       0.96      0.93      0.94        83

    accuracy                           0.94       153
   macro avg       0.94      0.94      0.94       153
weighted avg       0.94      0.94      0.94       153
```

8. Final results table and exported figures

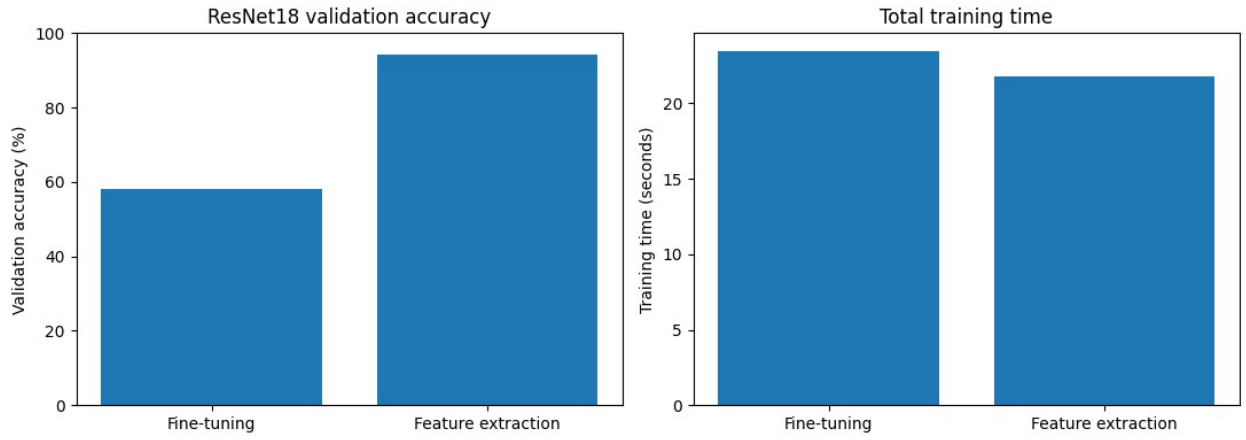
All figures and CSV files are saved under `results/q3/`. In Colab, download this folder or its individual files before the session expires. The saved notebook also retains inline figures when committed after execution.

```
out = Path('results/q3')
out.mkdir(parents=True, exist_ok=True)
history.to_csv(out/'resnet18_training_history.csv', index=False)
```

```
summary = history[history.epoch == EPOCHS].copy()
summary['training_seconds'] = summary['model'].map({'Fine-tuning':
fine_seconds,
                                                    'Feature
extraction': feature_seconds})
summary = summary[['model', 'train_accuracy', 'val_accuracy',
'train_loss',
                   'val_loss', 'training_seconds']]
summary.to_csv(out/'resnet18_summary.csv', index=False)
print(summary.to_string(index=False, float_format=lambda x:
f'{x:.4f}'))

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].bar(summary['model'], summary['val_accuracy'])
axes[0].set_ylabel('Validation accuracy (%)')
axes[0].set_ylim(0, 100)
axes[0].set_title('ResNet18 validation accuracy')
axes[1].bar(summary['model'], summary['training_seconds'])
axes[1].set_ylabel('Training time (seconds)')
axes[1].set_title('Total training time')
fig.tight_layout()
fig.savefig(out/'accuracy_and_training_time.png', dpi=180,
bbox_inches='tight')
plt.show()
print('Saved to:', out.resolve())
```

```
             model  train_accuracy  val_accuracy  train_loss  val_loss
training_seconds
       Fine-tuning         85.2459       58.1699      0.3304    3.3787
23.4673
Feature extraction         98.7705       94.1176      0.0730    0.1594
21.8045
```




```
Saved to: /content/EN3160-Neural-Networks/results/q3
```

Discussion – Question 3: ResNet18 Transfer Learning

In this experiment, two transfer learning approaches were compared using an ImageNet-pretrained ResNet18 model to classify ants and bees from the Hymenoptera dataset. In the fine-tuning approach, all layers of the pretrained network were updated during training, while in feature extraction, the pretrained layers were frozen and only the final classification layer was trained. Both models were trained for 10 epochs using the same dataset, batch size, optimizer, and learning rate.

After 10 epochs, the fine-tuning approach achieved a training accuracy of **96.72%** and a validation accuracy of **78.43%**, whereas feature extraction achieved **98.77%** training accuracy and **94.12%** validation accuracy. Feature extraction therefore performed considerably better, with a validation accuracy improvement of approximately **15.69 percentage points**.

The training and validation curves show that fine-tuning improved its training performance over the 10 epochs, but its validation accuracy fluctuated noticeably. Although the training loss decreased to 0.0945, the final validation loss remained relatively high at 0.5509. The large difference between training and validation accuracy, approximately 18.29 percentage points, suggests that the fine-tuned model did not generalize well to the validation images. In contrast, feature extraction produced more stable validation results, with a final validation loss of 0.1594 and an accuracy gap of only 4.65 percentage points.

The confusion matrices and classification reports also support these observations. Fine-tuning achieved a recall of 93% for ants but only 66% for bees, suggesting that it had more difficulty identifying bee images correctly. Feature extraction performed more consistently across both classes, achieving recall values of 96% for ants and 93% for bees.

One possible explanation is the relatively small size of the Hymenoptera training dataset, which contains only 244 images. Updating all the parameters of ResNet18 on such a limited dataset may disturb some of the useful features learned during ImageNet pretraining. By freezing the pretrained backbone, feature extraction preserves these features and only learns a new classifier for the two target classes. This may explain its better generalization in this experiment.

Feature extraction was also slightly faster, taking approximately **23.40 seconds**, compared with **26.00 seconds** for fine-tuning. It required training only 1,026 parameters, whereas fine-tuning updated all 11.18 million parameters.

Overall, feature extraction was the more effective approach under the conditions tested, achieving higher validation accuracy, lower validation loss, and a shorter training time. However, this does not mean that feature extraction is always better than fine-tuning. With a larger dataset, data augmentation, or a smaller learning rate for the pretrained layers, fine-tuning might achieve better results. The experiment demonstrates that preserving pretrained features can be particularly useful when working with a limited amount of training data.